

Leading a Team in the Age of AI

Metrics, adoption, and governance for engineering managers

Florian Bruniaux

May 2026

v1.0.0

Guide v3.40.0



Table of contents

1	The Measurement Problem	3
1.1	What Anthropic Data Confirms	3
2	The DORA Foundation	4
2.1	Deployment Frequency	4
2.2	Lead Time for Changes	4
2.3	Change Failure Rate	5
2.4	Mean Time to Recovery (MTTR)	6
2.5	The 2025 DORA Evolution	6
3	Beyond DORA: The SPACE Framework	7
3.1	Satisfaction and Well-being	7
3.2	Performance	7
3.3	Activity	7
3.4	Communication and Collaboration	8
3.5	Efficiency and Flow	8
3.6	The Velocity Trap	8
4	AI-Specific Metrics	9
4.1	% AI-Assisted Code	9
4.2	CFR: AI Code vs. Manual Code	9
4.3	Review Time: AI PRs vs. Manual PRs	9
4.4	Developer Code Understanding	10
4.5	PR Comprehension Time	10
5	Product Metrics	11
5.1	Time-to-Value	11
5.2	Feature Adoption Rate	11
5.3	Bug Escape Rate	11
5.4	Feature CSAT	12
6	By Team Size	13
6.1	5-Person Team	13
6.2	25-Person Team	14
7	Vanity Metrics to Drop	17
8	The 4-Question Test	20
9	Tooling	21
10	Implementation Roadmap	23
10.1	Phase 1 (Weeks 1-2): Instrument DORA	23
10.2	Phase 2 (Weeks 3-4): Baseline and Targets	23
10.3	Phase 3 (Month 2+): Product and AI Metrics	23
11	Summary: The Complete Stack	24
12	See Also	25
13	The Series	26

 Who is this whitepaper for?

Primary audience: Engineering Managers, Tech Leads, CTOs

Prerequisites: Claude Code in active use by the team

Reading time: 25 min

Key takeaways:

- The 4 DORA metrics recalibrated for an AI context (and why they behave differently)
- The 5 AI-specific signals that standard frameworks don't cover
- The minimal stack by team size: 5 tools for 5 people, not 20
- The 6 vanity metrics to drop starting tomorrow

In 5 lines: AI boosts team velocity, and the numbers are real. The risk is measuring the wrong thing. Sprint velocity, commits, lines of code: these indicators rise automatically when you adopt AI tools, but they tell you nothing about actual code quality or pace sustainability. This whitepaper gives the metrics stack that works when AI generates 70 to 90% of your output, with tools, benchmarks, and an implementation roadmap.

1 The Measurement Problem

Adopting AI in development changes delivery speed fast enough to break most existing benchmarks. A team that shipped 2 features per sprint in 2022 can now ship 6, with AI generating 70 to 90% of the code. That looks like a win on every traditional dashboard, and it may genuinely be a win. Or velocity is masking superficial reviews, skill atrophy, and growing technical debt that nobody fully understands.

The core tension: activity metrics rise immediately when you adopt AI tools, but quality and maintainability signals are slower and harder to instrument. Sprint velocity, daily commits, lines written: everything goes up. Production bug rate, PR comprehension time, developer confidence: those are the ones that will tell you the truth 6 months from now.

This guide gives engineering managers, tech leads, and CTOs a practical metrics stack: the DORA foundation recalibrated for an AI context, human factors through SPACE, and the AI-specific signals that standard frameworks don't cover.

1.1 What Anthropic Data Confirms

Anthropic's internal data (January 2026) is clear: **+67% PRs merged per engineer per day**, with 70 to 90% of shipped code AI-assisted. Not a marketing projection, not a controlled demo: production numbers from a mature engineering team using Claude Code in real conditions.

The practical consequence: "Medium" on DORA benchmarks is no longer a credible target for a team using AI tools at full capacity. If your team stagnates at "Medium" on deployment frequency or lead time after AI adoption, the constraint is in your processes and pipeline, not in developer productivity. Adjust your targets accordingly.

2 The DORA Foundation

DORA (DevOps Research and Assessment) measures the health of your delivery system, not individual contributors. That distinction matters: it keeps metrics conversations focused on process improvement rather than surveillance. It's also the most validated framework in the domain, backed by years of research across thousands of organizations.

2.1 Deployment Frequency

What it measures: How often you deploy to production.

How to measure: Count production deployments per day, week, or month. Most CI/CD tools expose this directly (GitHub Actions, CircleCI, Vercel, etc.).

2024 benchmarks:

Tier	Frequency
Elite	Multiple times per day
High	Once per day to once per week
Medium	Once per week to once per month
Low	Less than once per month

In an AI context: AI accelerates feature development, so your cadence should increase, provided your pipeline keeps up. If deployment frequency stays flat after widespread AI adoption, the bottleneck is downstream: staging environments, manual QA, review throughput. AI gives you more PRs to merge; it doesn't automatically improve the rest of the pipeline.

Warning signal: deployment frequency rising while Change Failure Rate also rises. That's AI-accelerated code that isn't being reviewed seriously.

2.2 Lead Time for Changes

What it measures: Time between a commit and that code in production.

How to measure: Timestamp at commit, timestamp at deployment. The delta is your lead time. Tools like LinearB and Faros.ai automate this from your CI/CD pipeline.

2024 benchmarks:

Tier	Lead Time
Elite	Less than 1 hour
High	1 hour to 1 week
Medium	1 week to 1 month
Low	More than 6 months

In an AI context: AI reduces coding time but has limited effect on non-code segments of lead time. PR review, staging validation, context-switching delays, and deployment windows are largely unchanged by AI assistance. If your lead time doesn't improve alongside AI adoption, the constraint is in review velocity or pipeline automation, not in coding.

Diagnosis: Map your stages explicitly (coding time, waiting for review, waiting for staging, deployment window) to know where the lever is.

2.3 Change Failure Rate

What it measures: Percentage of deployments that cause a production incident or require a rollback.

How to measure: (Failed deployments) / (Total deployments). “Failed” means requiring a hotfix, rollback, or incident response. Define that clearly before measuring.

2024 benchmarks:

Tier	Rate
Elite	0-5%
High	5-10%
Medium	10-15%
Low	More than 15%

In an AI context: This is the metric most at risk when AI adoption outpaces review discipline. AI generates syntactically correct and structurally plausible code that can still have subtle behavioral errors. Teams that treat AI-generated PRs as “lower risk” and rubber-stamp them see their CFR rise progressively over 6 to 12 months.

Recommendation: Track CFR separately for AI-generated code and manually written code. If the AI CFR is significantly higher, your review process needs reinforcement, not your AI tools.

2.4 Mean Time to Recovery (MTTR)

What it measures: How long it takes to restore service after a production incident.

How to measure: Time between the incident alert and service restored. Track in your incident management system.

2024 benchmarks:

Tier	MTTR
Elite	Less than 1 hour
High	Less than 1 day
Medium	1 day to 1 week
Low	More than 1 week

In an AI context: AI genuinely helps here, provided observability is already in place. AI-assisted diagnosis can significantly reduce time to root cause when the model has access to error logs, stack traces, and codebase context. But AI diagnosis is only as good as the signals it can read. Sequencing matters: instrument first, then wait for MTTR improvement from AI.

2.5 The 2025 DORA Evolution

The DORA 2025 report made a significant methodological change: the four-tier model (Elite/High/Medium/Low) has been abandoned. DORA now identifies **7 organizational archetypes** covering expanded dimensions: throughput, stability, team performance, product performance, individual efficiency, time on value work, friction, and burnout.

The consequence for teams: stop chasing “Elite” as an endpoint. “Elite” on deployment frequency can coexist with burnout and high friction. The new model pushes you to identify your archetype and improve your weakest dimensions rather than optimizing metrics where you’re already strong.

3 Beyond DORA: The SPACE Framework

DORA measures the delivery system; SPACE measures the people inside it. Both are necessary, and neither is sufficient alone.

SPACE was developed by researchers from GitHub, Microsoft, and the University of Victoria (published 2021). It covers five dimensions.

3.1 Satisfaction and Well-being

Are developers satisfied with their work, their tools, and their processes? Are there burnout signals?

Measure with: a quarterly developer experience survey (5 to 8 questions, anonymous). Track the trend over time, not the absolute score. A team at 3.2/5 moving to 3.8/5 over 6 months is in a better position than a team stuck at 4.0/5.

3.2 Performance

Does the work delivered function as intended? Does it meet quality and reliability expectations?

Measure with: Change Failure Rate (overlap with DORA), Bug Escape Rate (bugs reaching production divided by total bugs), and customer satisfaction on specific features.

3.3 Activity

What volume of work is produced?

Measure with: deployment frequency, throughput (features delivered per cycle), PR merge rate.

Critical note: Activity is the easiest dimension to measure and the easiest to optimize perversely. High commit counts, high PR volumes, high deployment frequency can coexist with very low actual value delivered. Activity metrics are inputs, not outcomes.

3.4 Communication and Collaboration

Is knowledge flowing? Are teams unblocked and connected?

Measure with: PR review latency (time between opening and first review), cross-team dependency resolution time, new contributor onboarding time.

3.5 Efficiency and Flow

Can developers do deep work without constant interruption? What friction exists in the development process?

Measure with: self-reported flow state frequency (in your developer survey), context-switching frequency, unplanned versus planned work ratio.

3.6 The Velocity Trap

Teams can reach “High” on DORA deployment frequency while scoring poorly on satisfaction, well-being, and efficiency. More deployments, but developers working evenings to meet sprint commitments, skipping design discussions because AI makes coding fast enough to skip planning, accumulating cognitive debt from AI code reviews they don’t fully understand.

SPACE captures these signals where DORA doesn’t. Using both frameworks together gives you the complete picture, and a divergence (DORA strong, SPACE weak) is an early warning signal of future DORA degradation: exhausted teams eventually deliver more slowly.

Recommended cadence

DORA: monthly leadership review. System health, delivery pipeline performance.

SPACE (satisfaction and efficiency dimensions): quarterly review. Human health, sustainable pace, skill development.

4 AI-Specific Metrics

Standard frameworks weren't designed with AI-assisted development in mind. These metrics fill the gap.

4.1 % AI-Assisted Code

The proportion of committed code that was AI-generated or AI-assisted. Available in Anthropic Contribution Metrics (Team and Enterprise plans), the GitHub Copilot dashboard, and similar tools for other AI coding assistants.

Why it matters: Provides context for everything else. A 5% CFR increase is a different signal if AI assists 10% of your code versus 80%. Track this as the denominator for all quality metrics. This number typically rises over time as adoption grows; benchmark it quarterly.

4.2 CFR: AI Code vs. Manual Code

Separate your Change Failure Rate by code origin: AI-generated commits versus manually written commits. Most enterprise AI coding tools can tag commits or PRs.

If the AI CFR is within 2 to 3 percentage points of the manual CFR, your review process is working. If it's significantly higher, review discipline has dropped. If it's lower, AI tools may be genuinely improving code quality in your domain.

4.3 Review Time: AI PRs vs. Manual PRs

Compare the average review time (opening to merge) for AI-generated PRs versus manually written PRs. If AI PRs are merged significantly faster than manual ones, you may have a rubber-stamping problem.

AI-generated code requires at least as much scrutiny at review as manually written code, possibly more: it can be wrong in non-obvious ways with high apparent confidence. A 30% faster review cycle for AI PRs is a yellow signal worth investigating.

4.4 Developer Code Understanding

A binary qualitative signal: during a PR review, can the author explain their AI-generated code in their own words, not just what it does but why it does it that way?

Track this informally through your code review culture. If reviewers start noticing that authors can't explain their AI-generated submissions, that's a skill atrophy signal that will manifest in higher CFR and longer MTTR 6 to 12 months from now.

4.5 PR Comprehension Time

A proxy for code clarity and maintainability: ask reviewers to report how long it took them to understand what a PR does (before they can assess whether it's correct). Track the median across your team.

Rising comprehension time suggests code is becoming more complex or less well-organized over time, regardless of who wrote it. AI-generated code can inflate this metric by producing syntactically dense implementations that are harder to reason about than simpler, more explicit alternatives.

5 Product Metrics

Engineering metrics measure how code is built. Product metrics measure whether the code actually solves the right problems. Most engineering teams track the former and leave the latter entirely to product managers. That creates a blind spot where a team can ship fast, with strong DORA scores, while the product drifts away from user needs.

5.1 Time-to-Value

How long does it take a new user to reach their first success with your product? Define “first success” concretely: first task completed, first item saved, first report generated.

Track this as a median across user cohorts, and watch for regressions after major releases. AI can accelerate your feature delivery rate without improving the new user experience, or even degrading it.

5.2 Feature Adoption Rate

Among users who could use feature X, what percentage actually uses it within 14 days of release? A feature delivered on time with strong DORA metrics but that nobody uses is still a failed feature.

Segment by user cohort (new versus existing users, different pricing tiers) to distinguish adoption problems from discoverability problems.

5.3 Bug Escape Rate

Bugs found in production divided by total bugs (pre-production bugs plus production bugs).

Formula: $\frac{\text{bugs_in_prod}}{(\text{bugs_before_prod} + \text{bugs_in_prod})}$.

If your Bug Escape Rate exceeds 20%, your QA and review processes are systematically failing to catch problems before they reach users. With AI-assisted development, this metric deserves close monitoring: faster code generation combined with less rigorous reviews can push Bug Escape Rate up even when the absolute number of bugs stays flat.

5.4 Feature CSAT

Customer satisfaction score tied to specific features, not the product as a whole. More actionable than NPS: instead of “how likely are you to recommend us,” ask “how useful was feature X for accomplishing task Y” on a 1-to-5 scale.

NPS is useful for brand-level sentiment but too lagging and too broad to drive development priorities. Feature CSAT gives you a signal within 2 to 4 weeks of a release rather than 6 to 12 months.

6 By Team Size

Different team sizes have different tolerances for measurement overhead. A 5-person team spending 20% of its time on metrics infrastructure is making a bad trade-off. A 25-person team without automated DORA tracking is navigating blind.

6.1 5-Person Team

Metrics to track:

Metric	How	Frequency
Deployment frequency	Manual count or from CI	Weekly
Cycle time (commit to prod)	Linear or GitHub time-stamp diff	Per PR, reviewed monthly
Time-to-value (north star product metric)	Analytics tool (Mixpanel, Amplitude, PostHog)	Monthly
Production bugs per month	Count in your issue tracker	Monthly
Developer satisfaction	Anonymous 5-question form	Quarterly

Tooling: GitHub Insights plus a shared spreadsheet. No need for a dedicated dashboard at this size; the overhead isn't worth it. What matters is the discipline of reviewing these numbers in a monthly retrospective, not the precision of tooling.

The 'we're too small' trap

The instinct at this size is often to skip metrics entirely (“we’re too small, we know each other, we talk every day”). Resist that. The value of metrics at 5 people is the discipline, not the visibility. Naming a north star metric and checking it every month forces conversations that daily standups don’t naturally trigger.

6.2 25-Person Team

Metrics to track:

Metric	How	Frequency
All 4 DORA metrics	LinearB or Faros.ai automated	Weekly (automated)
Cycle time by squad (not global)	Same tooling, segmented	Weekly
Bug Escape Rate	Issue tracker plus deployment markers	Monthly
Feature CSAT	In-app survey on key features	Per release
% AI-assisted code	Anthropic Contribution Metrics	Monthly
Developer satisfaction	Anonymous 8-question survey	Quarterly
PR review time	GitHub Analytics / LinearB	Weekly
Time-to-value	Analytics tool	Monthly

Tooling: LinearB or Faros.ai for DORA automation, GitHub Analytics for AI contribution data, PostHog or Amplitude for product metrics.

Avoid global averages

At 25 people, global averages hide squad-level problems. A team with 3 squads where 80% of incidents come from one squad will show a “Medium” global CFR and miss the signal entirely. Track DORA by squad, not just by organization.

PR review time is particularly revealing at this scale. When the review median exceeds 24 hours, it creates context-switching overhead: developers move to other tasks while waiting, then have to reload context when the review comes back. That reload cost

doesn't show up in any standard metric, but it accumulates across dozens of PRs per week.

7 Vanity Metrics to Drop

Drop	Replace with	Why
Sprint velocity	Cycle time + deployment frequency	Velocity is manipulable in 2 sprints by changing estimation practices. Cycle time is harder to game.
Lines of code	Bug Escape Rate	LOC measures output volume. With AI, LOC rises automatically. Bug Escape Rate measures output quality.
NPS alone	CSAT + time-to-value	NPS is a lagging brand signal, not an engineering steering metric. Feature CSAT is actionable within weeks.
Commits per day	Lead Time for Changes	Commit frequency measures activity. Lead time measures whether that activity actually delivers value.
Story points	Throughput (features delivered)	Points are defined relative to the team's own baseline and are manipulable by repointing.
Code coverage rate	Mutation testing score + Bug Escape Rate	Coverage tells you tests exist. Mutation testing tells you tests actually deliver value.

Note on story points in an AI context: if AI generates boilerplate and scaffolding automatically, the effort to implement a “3-point story” has dropped considerably. Teams that haven’t recalibrated their pointing will see velocity increases that reflect tool efficiency, not team capacity.

8 The 4-Question Test

Before adding a metric to your stack, run it through these four questions.

1. Can you act on it within 2 weeks?

If the answer is no, it's a reporting metric, not a steering metric. Reporting metrics belong in quarterly investor decks, not weekly team reviews. "Total API calls since launch" is a reporting metric. "API error rate over the last 7 days" is a steering metric.

2. Does it explain the why, not just the what?

"Churn is at 5%" tells you nothing. "80% of churned users never completed their first workflow" tells you where to look. When evaluating a metric, ask: if this number moves, do I know what to investigate?

3. Is it correlated with a business outcome?

Deployment frequency correlates with revenue in fast-iteration SaaS products. PR review time correlates with developer satisfaction and retention. Feature CSAT correlates with expansion revenue. Lines of code correlate with nothing important.

4. Can it be measured automatically?

If collecting the metric requires manual work, it will be abandoned within 3 months when load increases. Automate or eliminate.

The rule: fewer than 3 "yes" answers, drop the metric. A stack of 5 rigorous metrics is more useful than a stack of 20 poorly defined ones.

9 Tooling

Tool	What it does	Best for	Notes
LinearB	DORA automation + cycle time, connected to GitHub + Jira	25+ people	Strong DORA dashboards out-of-box, solid PR analytics
Faros.ai	DORA + custom engineering dashboards, open-source core	25+ people	More configurable than LinearB, heavier setup
GitHub Analytics (Anthropic)	AI contribution metrics (% AI-assisted code, PR attribution)	Any Claude Code team	Enterprise/Team plan required
Sleuth	Deployment tracking + change failure rate, DORA-focused	10+ people	Lightweight, CI/CD-focused, no bloat
Axify	Full engineering metrics suite, DORA + flow + team health	15+ people	Canadian startup, strong SPACE coverage
GitHub Insights	Basic activity metrics, free, native	Any size	Sufficient for 5-10 people, insufficient at scale
Spreadsheet	Manual tracking, always works, zero setup	Fewer than 10 people	The right tool when automated setup overhead isn't justified

No tool automatically surfaces the AI-specific metrics described above (CFR by code origin, review time comparison, comprehension signals). That requires either custom dashboards

built on your CI/CD data or manual tracking. GitHub Analytics covers % AI-assisted code; the rest you'll need to wire up yourself.

Avoid tool proliferation. A team with LinearB, Jira, GitHub Insights, and two separate analytics tools will spend more time reconciling numbers than acting on them. One DORA tool, one product analytics tool, GitHub Analytics for AI-specific data.

10 Implementation Roadmap

The most common failure mode in metrics programs is trying to instrument everything at once.

10.1 Phase 1 (Weeks 1-2): Instrument DORA

Connect your CI/CD pipeline to a metrics tool. For most teams, that means connecting GitHub Actions (or equivalent) to LinearB, Faros, or Sleuth. Automate Deployment Frequency and Lead Time first; they require the least manual setup. Change Failure Rate and MTTR require incident tracking to be in place, which takes a bit more time to configure.

Output: a live dashboard showing at minimum Deployment Frequency and Lead Time for Changes. Your first baseline numbers.

10.2 Phase 2 (Weeks 3-4): Baseline and Targets

Once you have 2 to 4 weeks of data, establish your real baseline. The temptation here is to compare immediately to industry benchmarks. Resist that. Set internal improvement targets first: “reduce Lead Time by 20% over the next quarter” is more actionable than “reach the High tier.” Your context (tech stack, deployment environment, team size, product type) affects what’s achievable more than any industry benchmark.

Launch your first developer satisfaction pulse (5 questions, anonymous, 10 minutes to build in Google Forms or Typeform). That’s your SPACE baseline.

10.3 Phase 3 (Month 2+): Product and AI Metrics

Once DORA is stable and automated, add product metrics (time-to-value, Feature CSAT) and AI-specific signals (% AI-assisted code, CFR by origin). These require more setup (product analytics instrumentation, PR tagging conventions) but are worth the investment once your DORA foundation is solid.

Review the full stack quarterly and prune ruthlessly. Any metric that hasn’t guided a decision in the last 3 months is a reporting metric disguised as a steering metric. Cut it.

11 Summary: The Complete Stack

Minimal stack by maturity phase

Phase 1: Foundation (months 1-2)

- Deployment frequency
- Lead Time for Changes
- Developer satisfaction (baseline)

Phase 2: Quality signal (months 2-4)

- Change Failure Rate (global + by AI/manual origin)
- MTTR
- Bug Escape Rate
- % AI-assisted code

Phase 3: Full steering (months 4+)

- Time-to-value
- Feature CSAT
- PR review time (AI vs. manual)
- PR comprehension time
- Developer satisfaction (recurring)

The temptation is to want everything from the start. A metrics stack, like a codebase, accumulates debt when it grows faster than the team's capacity to act on it. Build it in phases, prune regularly, and keep the 4-question test as a permanent filter.

12 See Also

- [Team Metrics: Reference Guide](#): full version with dashboards and additional resources
- [Session Observability](#): Claude Code session monitoring, cost tracking, usage patterns
- [AI Traceability](#): AI contribution auditing, attribution, and compliance
- [Team Deployment](#): Claude Code team adoption, CI/CD, champions program

13 The Series

This whitepaper is part of the **Claude Code Ultimate Guide** series, a comprehensive resource for mastering Claude Code from installation to production deployment.

12 whitepapers in the series:

- **WP00** Introduction & Fundamentals
- **WP01** Effective Prompts
- **WP02** Customization & Configuration
- **WP03** Production Security
- **WP04** Internal Architecture
- **WP05** Team Adoption
- **WP06** Privacy & Compliance
- **WP07** Condensed Reference Guide
- **WP08** Agent Teams & Sub-Agents
- **WP09** Learning with AI (UVAL)
- **WP10** AI Budget & ROI
- **WP11** Leading a Team in the Age of AI

Full series available at cc.bruniaux.com/whitepapers/